

Módulo 8: Acceso a datos y ORM 8

Contenidos clave

sqlite3 y drivers para PostgreSQL/SQL Server:

- driver: es el componente que permite que Python se comunique con una base de datos. Python -> Driver -> PostgreSQL. Python no puede simplemente hablar directamente con PostgreSQL; necesita una librería/driver que traduzca las operaciones. El driver se encarga de la comunicación; no es el ORM.
- SQLite es una base de datos muy sencilla que no necesita un servidor independiente. La base de datos normalmente es simplemente un archivo. Python ya incluye sqlite3. Python -> sqlite3 -> db. Es muy útil para pruebas, aplicaciones pequeñas y laboratorios.
- PostgreSQL y SQL Server: para bases de datos externas se necesitan sus respectivos drivers. PostgreSQL tiene opciones como psycopg (Python -> Driver de PostgreSQL -> PostgreSQL). Para SQL Server existen drivers/librerías específicas para comunicarse con ese motor.

SQLAlchemy Core y ORM; relaciones y consultas:

- SQLAlchemy Core: es una librería de Python para trabajar con bases de datos. Tiene dos enfoques principales:
 1. Core: SQLAlchemy Core permite trabajar con SQL y estructuras de bases de datos de una manera más programática. Aún con SQL administra la conexión y la ejecución.
 2. ORM: SQLAlchemy ORM (Object-Relational Mapping) Mapeo Objeto-Relacional. La idea es convertir; en lugar de trabajar directamente con filas SQL, se puede trabajar con objetos Python. Una clase puede representar una tabla, ejemplo: "user" y se puede crear un usuario como objeto para posteriormente guardarlo en la base de datos. Objeto Python -> ORM -> Fila en la tabla.
- Relaciones en ORM: las bases de datos normalmente tienen relaciones entre tablas. Para el laboratorio de este módulo sería User 1:N Order y Order 1:N OrderItem. Un user puede tener muchas Order y una Order puede tener muchos OrderItem. Lo anterior se puede representar en SQLAlchemy.
- Consultas con SQLAlchemy: una vez que se tienen los modelos se puede buscar información. Por ejemplo:

```
from sqlalchemy import select
stmt = select(User).where(User.name == "Ana")
usuarios = session.scalars(stmt).all()
```

Lo anterior conceptualmente equivale a:

```
SELECT *
FROM users
WHERE name = 'Ana';
```

Solo que estamos trabajando con objetos Python.

- CRUD:

→ C: Create -> Crear usuario.

```
usuario = User(name="Ana")
session.add(usuario)
session.commit()
```

→ R: Read -> Consultar usuario.

```
usuario = session.get(User, 1)
```

→ U: Update -> Modificar usuario.

```
usuario.name = "Ana López" session.commit()
```

→ D: Delete -> Eliminar usuario.

```
session.delete(usuario) session.commit()
```

- Transacciones: es un conjunto de operaciones que deben tratarse como unidad. Si algo falla, es preferible que todo se deshaga.

Migraciones con Alembic: permite crear y controlar migraciones de base de datos. Permite llevar un historial de cambios aplicado a la estructura de base de datos.

Introducción a MongoDB (Motor): MongoDB es una base de datos NoSQL orientada a documentos. En lugar de tabla -> filas -> columnas, trabaja con documentos similares a JSON. Motor es un driver/cliente de Python para trabajar de forma asíncrona con MongoDB.

Objetivos

- Modelar entidades y relaciones en ORM.

Se crearon 3 clases: User, Order y OrderItem, cada una representa una tabla de la base de datos. SQLAlchemy hace el mapeo mediante:

```
14 class User(Base):
45 class Order(Base):
79 class OrderItem(Base):
```

Las relaciones quedaron cómo:

- Un user puede tener muchas Order.
- Una Order puede tener muchos OrderItem.

Y se implementaron con:

```
38     orders: Mapped[list[Order]] = relationship(  
39         "Order",  
40         back_populates="user",  
41         cascade="all, delete-orphan",  
42     )
```

```
72     items: Mapped[list[OrderItem]] = relationship(  
73         "OrderItem",  
74         back_populates="order",  
75         cascade="all, delete-orphan",  
76     )
```

Las claves foráneas establecen la relación en la base de datos:

```
54         ForeignKey("users.id"),
```

```
88         ForeignKey("orders.id"),
```

- Gestionar migraciones y transacciones.

- Transacciones:

Se hizo uso de:

```
45     with session.begin():  
46         session.add(user)  
47  
48     session.refresh(user)  
49  
50     logger.info(  
51         "Usuario '%s' y Orden #%d creados con éxito.",  
52         user.username,  
53         order.id,  
54     )  
55  
56     return user
```

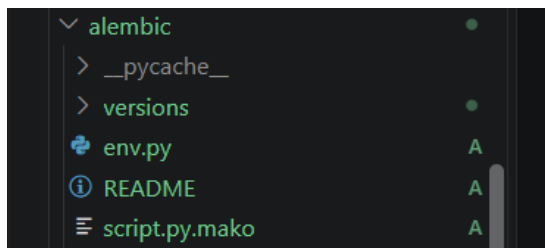
Dónde se crea/añade un usuario, orden e ítems.

- Migraciones:

Se inicializó Alembic:

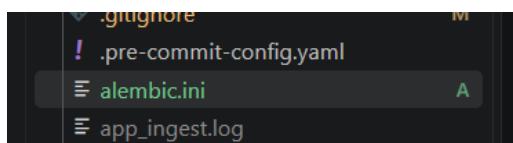
```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run alembic init alembic  
Creating directory C:\Users\Quiryat\Documents\Python\python-mastery-capstone\alembic ... done  
Creating directory C:\Users\Quiryat\Documents\Python\python-mastery-capstone\alembic\versions ... done  
Generating C:\Users\Quiryat\Documents\Python\python-mastery-capstone\alembic.ini ... done  
Generating C:\Users\Quiryat\Documents\Python\python-mastery-capstone\alembic\env.py ... done  
Generating C:\Users\Quiryat\Documents\Python\python-mastery-capstone\alembic\README ... done  
Generating C:\Users\Quiryat\Documents\Python\python-mastery-capstone\alembic\script.py.mako ... done  
Please edit configuration/connection/logging settings in C:\Users\Quiryat\Documents\Python\python-mastery-capstone\alembic.in  
i before proceeding.
```

Una vez iniciado, se realizó dentro del archivo alembic/[env.py](#) la importación de los modelos del laboratorio y se asignó el target_metadata, esto permite que Alembic conozca la estructura definida en nuestros modelos:



```
6 from labs.day08_orm_alembic.models import Base
21 target_metadata = Base.metadata
```

Posteriormente se configuró la URL de la base de datos en el archivo alembic.ini, dentro de la propiedad sqlalchemy.url para definir la ruta de la base de datos local SQLite:



```
89 sqlalchemy.url = sqlite:///app.db
```

Generación de la migración:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run alembic revision --autogenerate -m "init_schema"
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.schemas
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.tables
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.types
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.constraints
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.defaults
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.comments
INFO [alembic.autogenerate.compare.tables] Detected added table 'users'
INFO [alembic.autogenerate.compare.tables] Detected added table 'orders'
INFO [alembic.autogenerate.compare.tables] Detected added table 'order_items'
Generating C:\Users\Quiryat\Documents\Python\python-mastery-capstone\alembic\versions\068d0a758db3_init_schema.py ... done
```

Alembic detectó los modelos SQLAlchemy mediante Base.metadata y detectó las 3 tablas generando el archivo de migración 068d0a758db3_init_schema.py.

Posterior a ello aplicamos la migración:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run alembic upgrade head
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running upgrade -> 068d0a758db3, init schema
```

Lo anterior muestra que la base de datos ya tiene un historial controlado de cambios mediante Alembic.

Laboratorio

- Modelos User/Order/OrderItem, CRUD básico, migración Alembic y pruebas en SQLite en memoria.

Los modelos User / Order / OrderItem se implementaron con SQLAlchemy.

Se implementaron las 4 operaciones:

- Create: Creación de usuario junto con su orden e items.

```
128 session.add(user)
```

- Read: Consultar las órdenes de un usuario.

```
71 stmt = select(Order).join(User).where(User.username == "reader")
```

- Update: Modificar el estado.

```
99 order.status = "SHIPPED"
```

- Delete:

```
133 session.delete(user)
```

Configuración de SQLite en memoria: permite tener una base de datos temporal para pruebas.

```
9 DATABASE_URL = "sqlite:///memory:"
```

Pruebas automatizadas:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run pytest labs/day08_orm_alembic/test_crud.py -v
===== test session starts =====
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Quiryat\AppData\Local\pypoetry\Cache\virtualenvs\python-mastery-capstone-Sh3whZ1x-py3.14\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Quiryat\Documents\Python\python-mastery-capstone
configfile: pyproject.toml
plugins: anyio-4.15.1
collected 4 items

labs/day08_orm_alembic/test_crud.py::test_create_user_order_and_items PASSED [ 25%]
labs/day08_orm_alembic/test_crud.py::test_read_user_orders PASSED [ 50%]
labs/day08_orm_alembic/test_crud.py::test_update_order_status PASSED [ 75%]
labs/day08_orm_alembic/test_crud.py::test_delete_user_cascades_orders PASSED [100%]

===== 4 passed in 1.76s =====
```

La migración con Alembic tuvo el siguiente flujo:

[models.py](#) -> Base.metadata -> Alembic -> revisión --autogenerate -> init_schema.py -> upgrade head, detectando de manera correcta users orders y order_items por lo que la migración quedó creada y aplicada.